

TICKET BOOKING WEB APPLICATION FOR TECHNICAL FEST

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

V.RAMBABU (VTU25536)

*10211CS224
Full Stack Application Development
Slot : E1-Bucket 4*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF
SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

April, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "TICKET BOOKING WEB APPLICATION FOR TECHNICAL FEST" by V.RAMBABU (VTU25536) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.HAFIZUR RAHMAN

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

Signature of Course Coordinator

Dr.Sumithra M.

Assistant Professor (Senior Grade)

Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(V.Rambabu)

Date:06/05/2026

ABSTRACT

The Ticket Booking Web Application for Technical Fest is a full-stack web-based system developed to simplify and automate the process of event ticket booking during technical festivals. Traditional methods of ticket distribution often involve manual handling, long queues, and lack of real-time updates, which can lead to inefficiencies and user inconvenience. This application provides a centralized digital platform where users can easily browse fest events, view schedules, and book tickets online in a seamless manner.

Additionally, the system supports features such as user authentication, booking confirmation, and notification services to enhance reliability and user experience. By digitizing the ticket booking process, the application reduces manual effort, improves accuracy, and ensures transparency.

Overall, the system offers a scalable and efficient solution for managing technical fest events and ticket bookings.

Keywords:

Ticket Booking System, Event Management, Web Application, User Authentication, Online Reservation, Database Management, Internal Events, Automation

LIST OF FIGURES

1.1	Framework of the implemented Project	2
3.1	Backend Implementation	7
4.1	System Architecture of Ticket Booking Application	10
4.2	Data Flow Diagram of Ticket Booking System	11
5.1	Landing Page Interface	15
5.2	Authentication Page	15
5.3	User Dashboard	16
5.4	Admin Dashboard	16
5.5	Events Page	17
5.6	Ticket Booking Page	17
5.7	Payment Page	18
5.8	Ticket Pass	18

LIST OF TABLES

2.1	Comparative Analysis of Market Alternatives	4
3.1	System Technology Stack	9
5.1	Ticket Booking System Module Comparison	19
8.1	Mapping of Project to Complex Engineering Problem (CEP)	26
8.2	Mapping of Project to Complex Engineering Problem (CEP)	27
8.3	SDG Mapping of the Project	27

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CI	Continuous Integration
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
MVC	Model View Controller
ORM	Object Relational Mapping
TBDE	Ticket Booking Department Event
TX	Technical Fest Tickets
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	1
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	3
3 FRAMEWORK DESCRIPTION	5
3.1 Frontend Implementation	5
3.2 Backend Implementation	6
3.3 Database Implementation	8
3.4 System Technology Stack Overview	9
4 METHODOLOGIES	10
4.1 Project Architecture	10
4.2 Data Flow Diagram	11

4.3	Module 1: User Management	11
4.4	Module 2: Event Management	12
4.5	Module 3: Ticket Booking	12
4.6	Advantages of this Project	13
4.7	Disadvantages	13
5	RESULTS AND DISCUSSIONS	14
5.1	System Results	14
6	CONCLUSION AND FUTURE ENHANCEMENTS	20
6.1	Conclusion	20
6.2	Future Enhancements	20
7	SOURCE CODE	22
7.1	Source Code	22
8	Addressing Complex Engineering Problem and SDG	26
8.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	26
8.2	Mapping of the Project to Complex Engineering Activities (CEA) .	27
8.3	Sustainable Development Goals (SDG) Mapping	27
	References	27

Chapter 1

INTRODUCTION

1.1 Introduction

The Ticket Booking System for an Technical Fest is designed to provide an efficient and reliable solution for managing event registrations within an organization or academic department. Traditional ticket booking methods are often time-consuming and prone to errors, especially when handling many participants. This system offers a digital platform where users can view event details, check availability, and book tickets securely, while administrators can manage events and monitor bookings in real time.

1.2 Aim of the Project

The aim of this project is to develop a user-friendly web-based system for booking tickets for Technical Fests. It focuses on automating the registration process to reduce manual effort and errors.

1.3 Scope of the Project

The scope of the Ticket Booking System for an Technical Fest includes developing a web-based platform that allows users to view event details, check ticket availability, and book tickets online. It covers features such as secure user authentication, event creation, ticket management, and booking tracking by administrators. The system is intended for internal use within a department or organization, aiming to improve efficiency, reduce manual work, and ensure accurate and organized event

management.

1.4 Framework

The system framework consists of layers like user interface, application logic, and database.

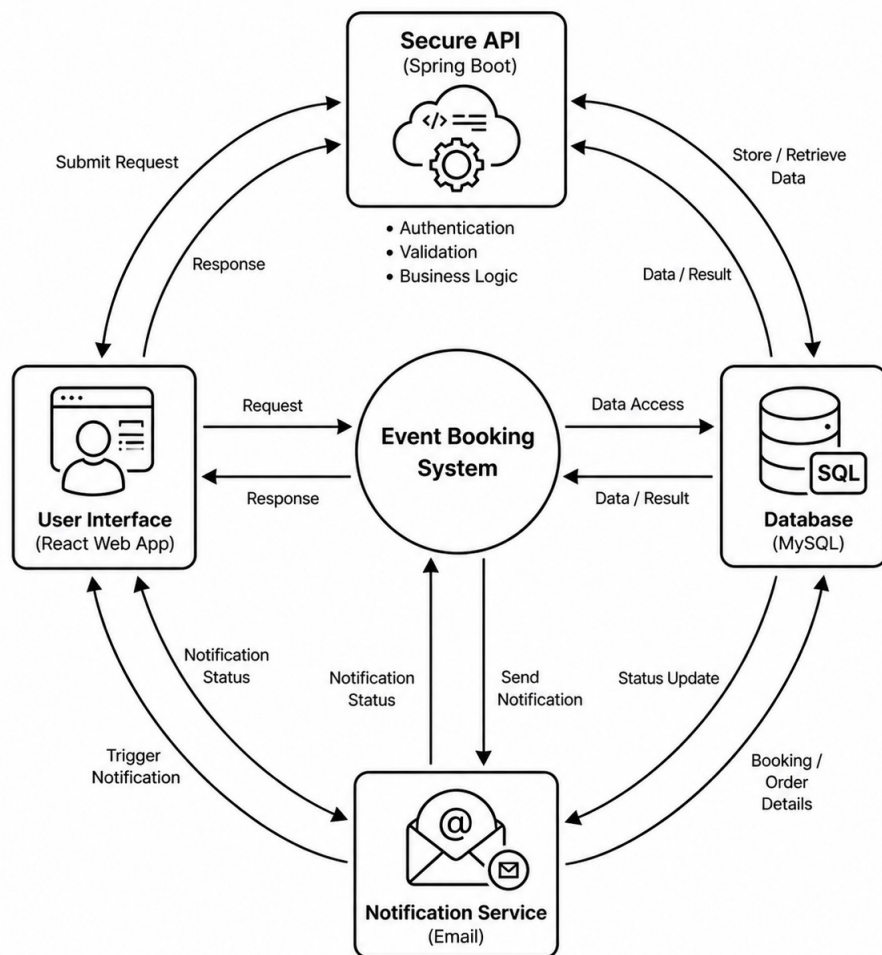


Figure 1.1: Framework of the implemented Project

Figure 1.1 shows the frame work of the implemented Project. The Ticket Booking Web Application follows a multi-layered architecture consisting of frontend, backend, service, and database layers. The frontend allows users to browse events and book tickets through an interactive interface, while the backend processes requests and handles business logic.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several ticket booking and event management applications are widely used in the market, such as BookMyShow, Eventbrite, Ticketmaster, and Paytm Insider [1]. Meta Platforms, Inc. (2024). *React: A JavaScript Library for Building User Interfaces*, React Official Documentation, 1(1), pp. 1–10. <https://react.dev/> These platforms provide essential features including real-time ticket booking, secure online payment integration, event discovery, and user-friendly interfaces enhance the overall user experience [2]. OpenJS Foundation (2024). *Node.js Runtime Environment Documentation*, Node.js Official Docs, 2(1), pp. 1–12. <https://nodejs.org/en/docs>

In addition to these, platforms like Eventzilla and Arlo Event Management offer advanced functionalities such as customizable event pages, attendee management, automated communication, and detailed reporting tools for organizers [3]. Oracle Corporation (2024). *MySQL Relational Database Management System*, MySQL Official Documentation, 3(2), pp. 1–15. <https://dev.mysql.com/doc/> These features help in efficiently managing large-scale events and tracking participant engagement [4]. Razorpay Software Private Limited (2024). *Online Payment Gateway Integration for Web Applications*, Razorpay Developer Documentation, 5(1), pp. 10–18. <https://razorpay.com/docs/>

Furthermore, systems like Booking Ninjas provide real-time tracking, and centralized dashboards, which significantly improve operational efficiency and reduce manual effort [5]. Stripe, Inc. (2024). *Secure Payment Processing and API Inte-*

gration, Stripe API Reference, 6(2), pp. 20–30. <https://stripe.com/docs/api> Most existing applications are primarily designed for large-scale public events such as concerts, movies, and conferences.

Table 2.1: Comparative Analysis of Market Alternatives

Feature	BookMyShow	Eventbrite	Townscript	Your Project
Event Creation	Yes	Yes	Yes	Yes
Ticket Booking	Yes	Yes	Yes	Yes
Internal Department Events	No	No	No	Yes
User Registration/Login	Yes	Yes	Yes	Yes
Online Payment Integration	Yes	Yes	Yes	No / Planned
QR Code / E-Ticket	Yes	Yes	Yes	Yes
Seat Selection	Yes	Limited	Limited	Yes
Event Analytics	Yes	Yes	Yes	Basic
Direct Communication (Admin-User)	No	Limited	No	Yes

Table 2.1 shows the Comparative analysis highlights that existing platforms like BookMyShow, Eventbrite, and Townscript provide common features such as event creation, ticket booking, user authentication, QR-based e-tickets, and analytics. However, these platforms are mainly designed for large-scale public events and lack support for internal department-level event management within institutions.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend development environment is set up using modern web technologies such as HTML, CSS, and JavaScript. Tools like code editors (e.g., VS Code) and web browsers are used for development and testing. Required libraries and frameworks, if any, are installed to enhance design and functionality.

3.1.2 Structure of the Project

The project follows a modular structure to ensure scalability, maintainability, and clear separation of concerns:

/public: Contains static assets such as index.html, logos, event banners, and other images used across the application.

/src/api: Includes configuration files (e.g., axios.js) that define the base URL and handle communication with the backend server.

/src/components: Contains reusable UI components such as Navbar, Sidebar, Event Cards, Ticket Forms, and QR Code generator.

/src/context: Manages global state using context APIs (e.g., AuthContext.js) for user authentication, session handling, and role-based access (Admin/User).

/src/pages: Includes the main pages of the application-

Home.js: Displays available events and general information.

EventDetails.js: Shows detailed information about a selected event.

Booking.js: Handles ticket booking, seat selection, and user inputs.

Dashboard.js: Admin panel for managing events, bookings, and users.

MyTickets.js: Allows users to view and download their booked tickets.

/src/utlis: Contains helper functions such as validation, date formatting, and ticket generation logic.

App.js: The root component that defines routing, navigation, and overall layout of the application.

3.1.3 Execution Procedure

The project is executed by opening the main HTML file in a web browser or running it through a local development server. Users can interact with the interface to view events, book tickets, and perform other actions, while developers can test and debug features during execution.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend ecosystem requires coordinated setup of multiple components to ensure smooth operation of the ticket booking and event management system:

Frontend Layer (React Application): Node.js (v14 or above) for running the development server and managing dependencies. npm or Yarn for package management. React.js framework for building a responsive and dynamic user interface.

Backend Layer (Spring Boot): Java Development Kit for application development and execution. Apache Maven for dependency management and project build lifecycle. Spring Boot for developing RESTful APIs and handling business logic.

Database Layer: MySQL (v8.0 or above) for storing user data, event details, and booking information. MySQL Workbench (optional) for database design and administration.

3.2.2 Structure of the Project

The backend project is organized into folders such as routes, controllers, and database

configurations. It includes files for handling user authentication, event management, and ticket booking logic. This structured approach helps in maintaining clean code and easy scalability of the application.

3.2.3 Execution Procedure

The backend server is started using appropriate commands (e.g., npm start or npm run dev). Once the server is running, it handles client requests, processes data, and communicates with the database. APIs are tested using tools like Postman to ensure proper functionality.

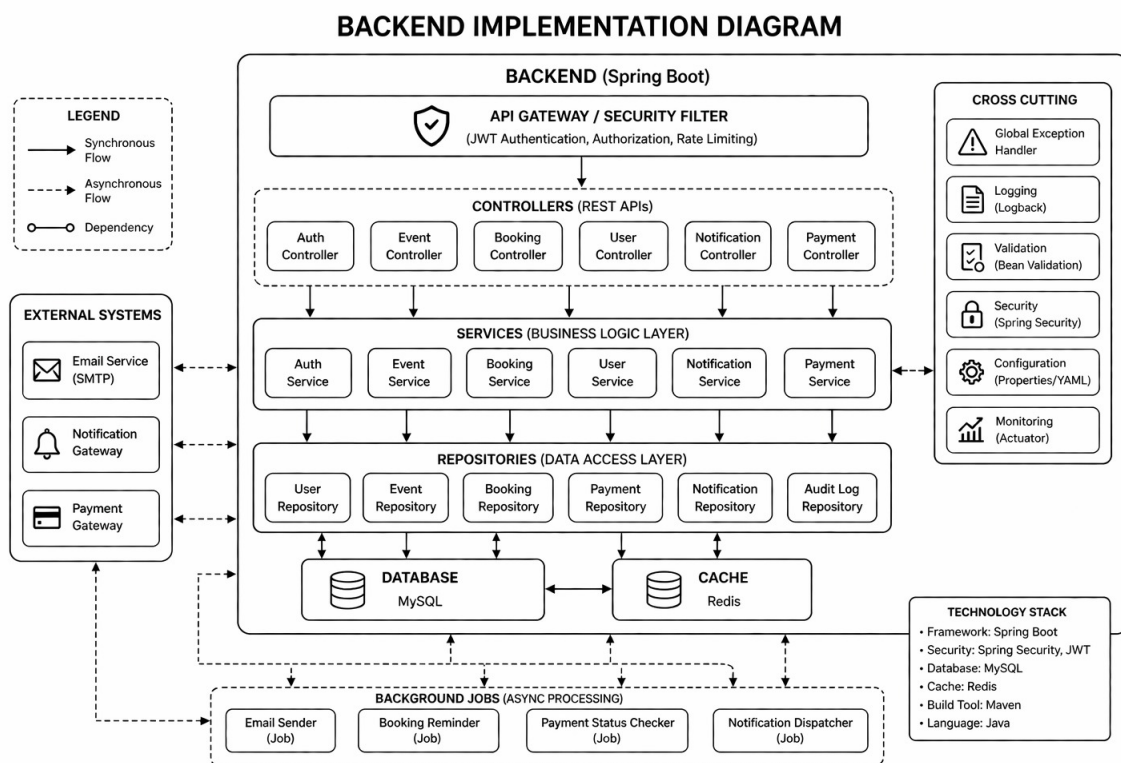


Figure 3.1: Backend Implementation

Figure 3.1 shows that the backend implementation of the Ticket Booking Web Application for Technical Fest is developed using a server-side framework such as Node.js with Express, which handles all core functionalities and business logic of the system. It manages user authentication, event creation and management, ticket booking processing, seat availability checks, and transaction handling.

3.3 Database Implementation

3.3.1 Database Configuration

The database is configured using a relational database management system such as MySQL. Necessary settings like database creation, user permissions, and connectivity with the backend are established. The system ensures secure and efficient storage of data related to users, events, and bookings.

3.1.2 Schema Structure

The database schema is designed to organize data into structured tables with proper relationships. It includes entities such as users, events, tickets, and bookings, connected through primary and foreign keys to maintain data integrity and consistency.

3.1.3 Tables created

The core functionalities are supported by the following primary tables:

Users: Stores user information including name, email, encrypted password, role (Admin/User), and contact details. It manages authentication and access control within the system.

Events: Contains details of all internal department events such as event name, description, date, time, venue, and organizer information.

Tickets: Stores ticket-related information including ticket ID, event ID, seat number (if applicable), and ticket type (general/VIP).

Bookings: Acts as a junction table between users and events, storing booking details such as booking ID, user ID, event ID, number of tickets, booking date, and status (Confirmed, Cancelled).

Payments: Maintains transaction records including payment ID, booking ID, payment method, amount, and payment status (Success, Failed, Pending).

3.4 System Technology Stack Overview

The following table provides a comprehensive summary of the frameworks and technologies utilized in the development of the Ticket Booking of an Internal Department Event system, categorized by their architectural role.

Table 3.1: System Technology Stack

Component	Technology
Frontend	React.js, HTML5, CSS3
Backend	Spring Boot, Spring Security
Database	MySQL
API Communication	Axios
Tools	Git, Postman, VS Code

Table 3.1 shows that the system technology stack for the Ticket Booking Web Application is designed to ensure performance, scalability, and ease of development. The frontend is built using React.js along with HTML5 and CSS3 to provide a dynamic, responsive, and user-friendly interface.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a layered architecture with frontend, backend, and database components. The frontend handles user interaction, the backend manages logic and processing, and the database stores all data. This structure ensures efficient, secure, and scalable system operation.

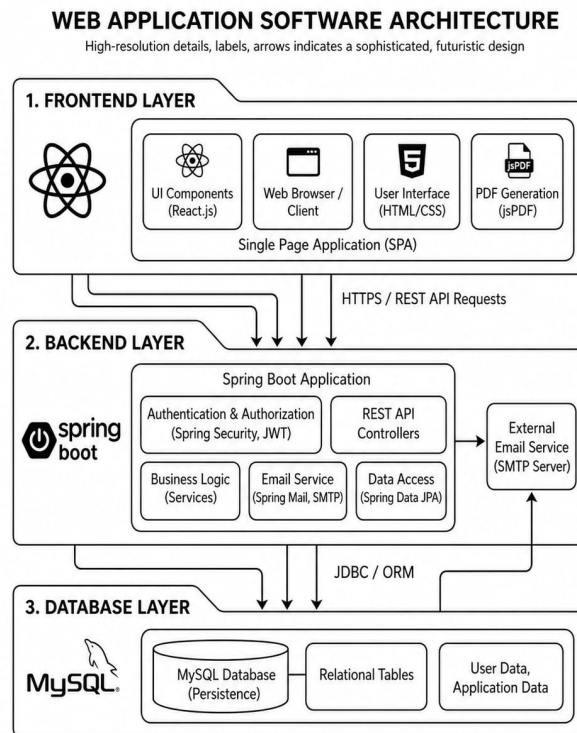


Figure 4.1: System Architecture of Ticket Booking Application

Fig 4.1 describes the three-tier architecture where the presentation layer (frontend) interacts with users, the application layer (backend) processes requests, and the data layer (database) stores all information securely.

4.2 Data Flow Diagram

The Data Flow Diagram shows how data moves within the system. User inputs are processed by the backend and stored in the database. The system then returns the required output, such as booking confirmation, to the user.

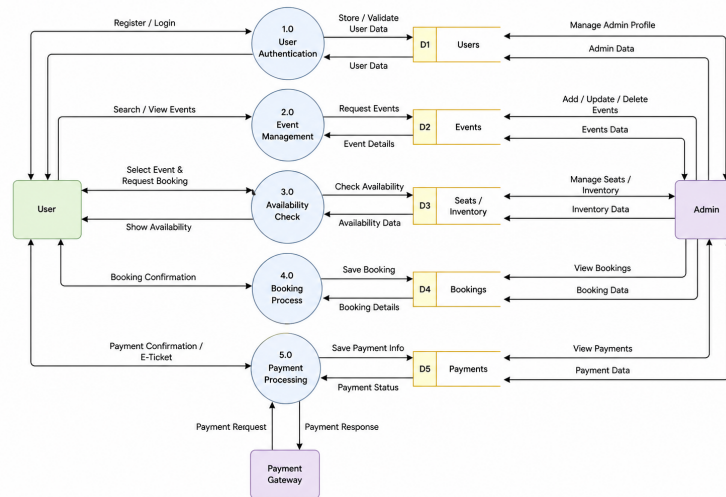


Figure 4.2: Data Flow Diagram of Ticket Booking System

Fig 4.2 illustrates the flow of data between users, the application, and the database, showing how booking requests are processed and confirmations are generated.

4.3 Module 1: User Management

This module handles user registration, login, and authentication. It ensures that only authorized users can access the system and perform booking operations.

Key Features:

- User registration with email verification
- Secure login with password encryption
- Role-based access (Admin/User)
- Profile management and password reset

4.4 Module 2: Event Management

This module allows administrators to create, update, and manage event details such as date, time, and ticket availability.

Key Features:

- Create new events with details
- Update event information
- Set ticket prices and quantity limits
- View event-wise booking statistics

4.5 Module 3: Ticket Booking

This module enables users to book tickets, check availability, and receive booking confirmation. It also prevents overbooking through proper validation.

Key Features:

- Real-time ticket availability check
- Multiple ticket selection
- Secure payment integration
- Instant booking confirmation with QR code

4.6 Advantages of this Project

The system simplifies ticket booking, reduces manual work, and improves efficiency. It ensures accurate data management and provides a user-friendly interface for both users and administrators.

- a) Time Saving:** Automates the booking process, reducing manual effort.
- b) Error Reduction:** Minimizes human errors and prevents overbooking.
- c) Easy Management:** Helps administrators manage events and bookings efficiently.

4.7 Disadvantages

The system depends on internet connectivity and may face technical issues if not maintained properly. Initial setup and development may require time and technical knowledge.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 System Results

The Ticket Booking Web Application for Technical Fest was successfully developed and tested. The system provides a user-friendly interface for managing event ticket bookings efficiently. The results demonstrate that the system reduces manual effort and improves user experience for festival attendees.

The application effectively manages booking records by storing them securely in the database, reducing the chances of data loss and duplication. The admin module proved useful in creating, updating, and monitoring events, as well as tracking ticket sales and user participation. The notification feature successfully delivered booking confirmations and updates, improving user engagement and communication.

However, certain limitations were observed, such as dependency on internet connectivity and basic security implementation. These can be improved in future versions by incorporating advanced security mechanisms, payment gateway integration, and mobile application support.

Overall, the results indicate that the developed system is reliable, user-friendly, and effective in managing technical fest ticket bookings digitally, providing a practical solution for modern campus event management.

The following screenshots illustrate the working of the system:

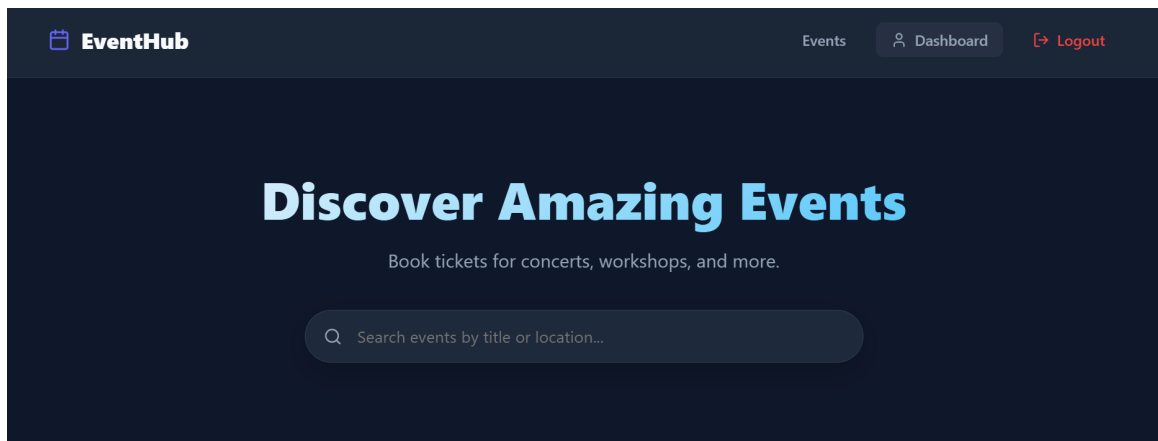


Figure 5.1: Landing Page Interface

Fig 5.1 describes the landing page which serves as the entry point for users, showcasing upcoming technical fest events, featured workshops, and key highlights of the festival.

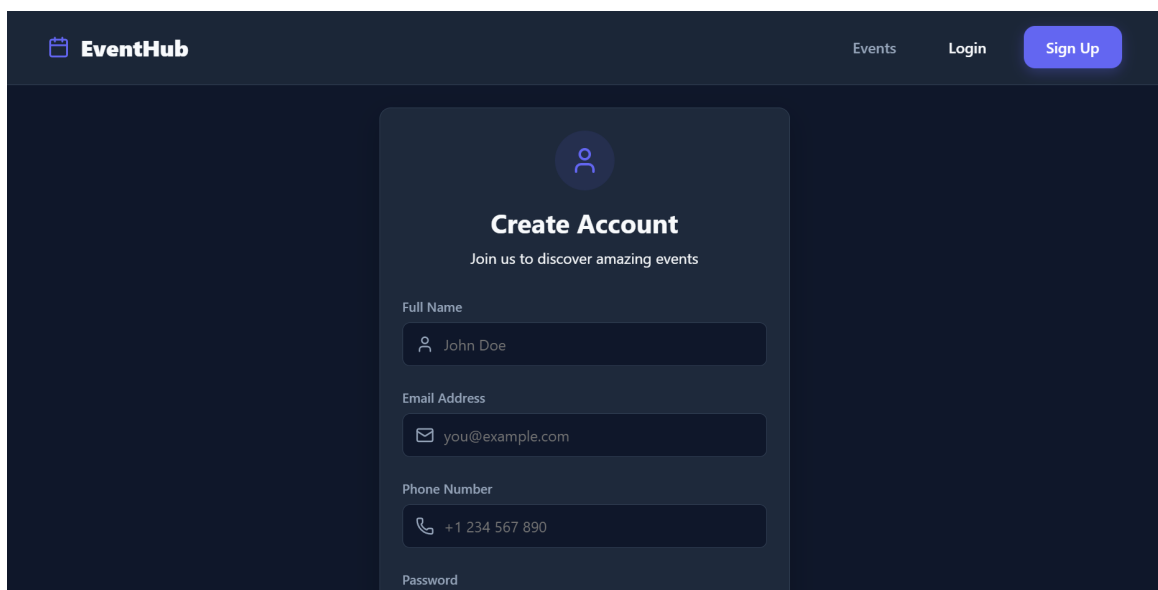


Figure 5.2: Authentication Page

Fig 5.2 describes the authentication page which allows users to securely login or register using their credentials. It ensures secure access and protects user data.

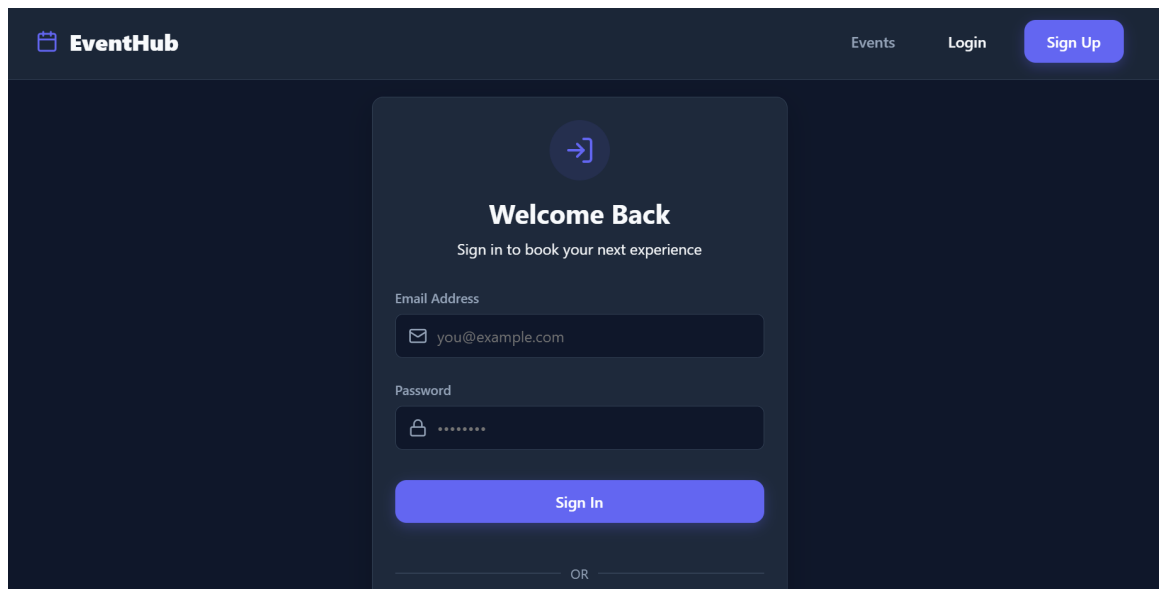


Figure 5.3: User Dashboard

Fig 5.3 describes the user dashboard which provides an overview of upcoming events, booked tickets, payment history, and personalized event recommendations.

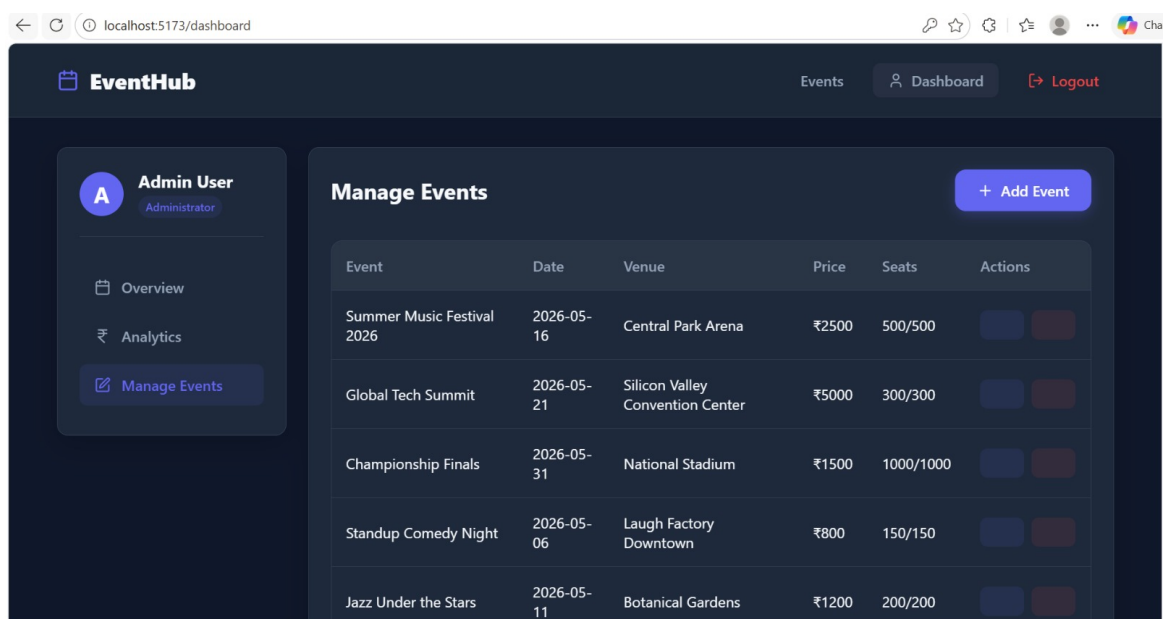


Figure 5.4: Admin Dashboard

Fig 5.4 describes the admin dashboard which enables administrators to manage events, view total bookings, track revenue, generate reports, and oversee user activities.

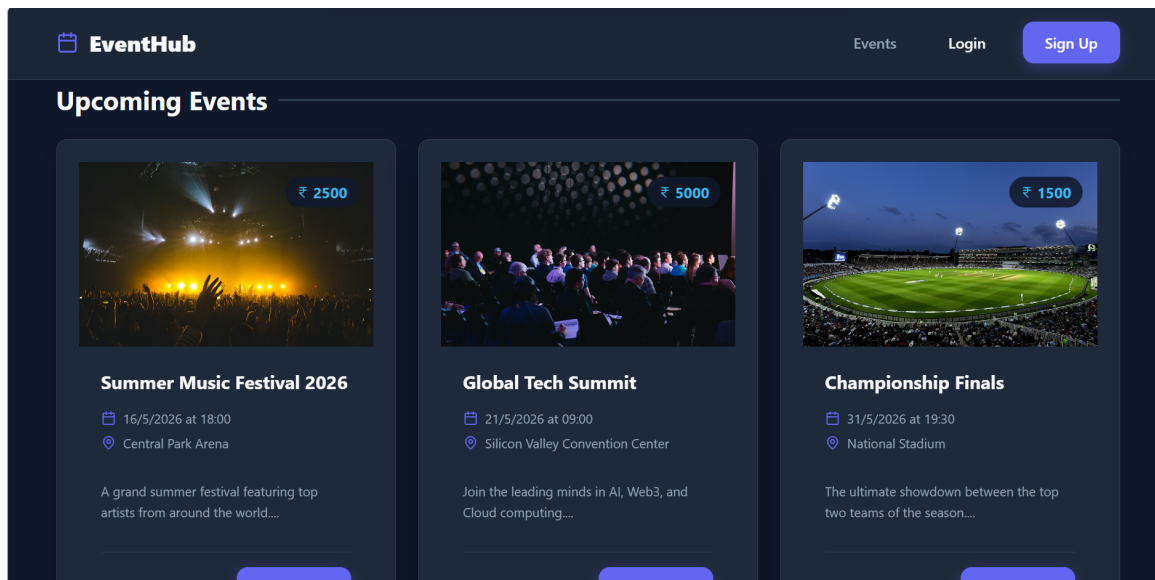


Figure 5.5: Events Page

Fig 5.5 describes the events page which displays all available technical fest events including workshops, competitions, seminars, and cultural programs with their schedules and prices.

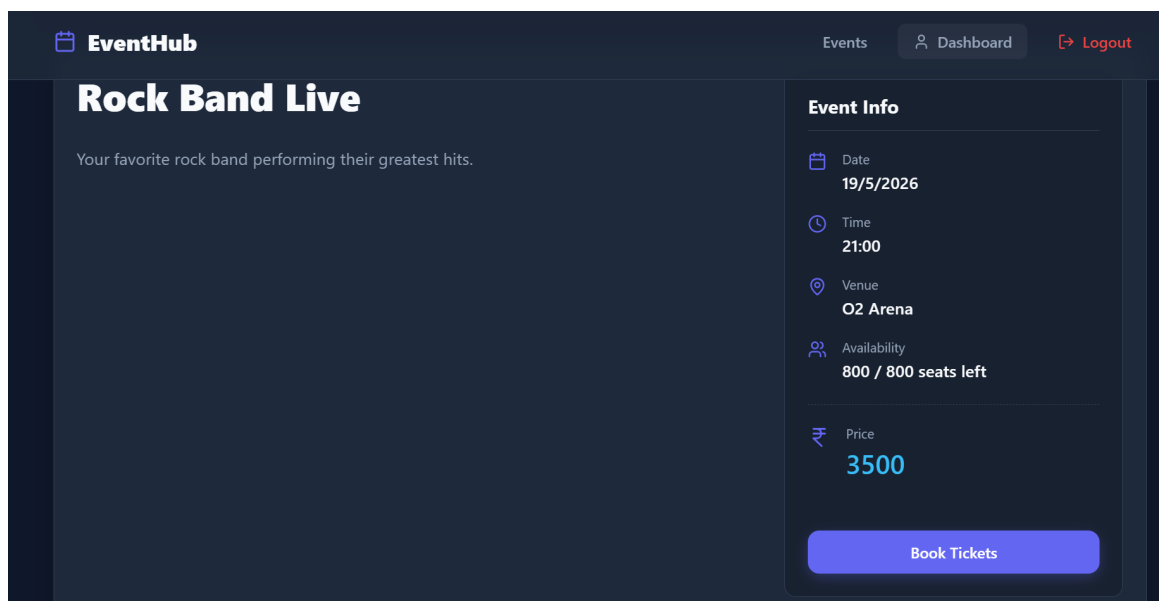


Figure 5.6: Ticket Booking Page

Fig 5.6 describes the ticket booking page which allows users to select event categories, choose number of tickets, apply promo codes, and proceed to payment.

EventHub Events Dashboard Logout

Complete Payment

Select Method

- Credit/Debit Card
- UPI App
- Net Banking

Payment Details

Card Number: 0000 0000 0000 0000

Expiry Date: MM/YY CVV: 123

Name on Card: John Doe

Pay ₹8000

Figure 5.7: Payment Page

Fig 5.7 describes the payment page which integrates secure payment gateways allowing users to complete transactions using credit/debit cards, UPI, or net banking.

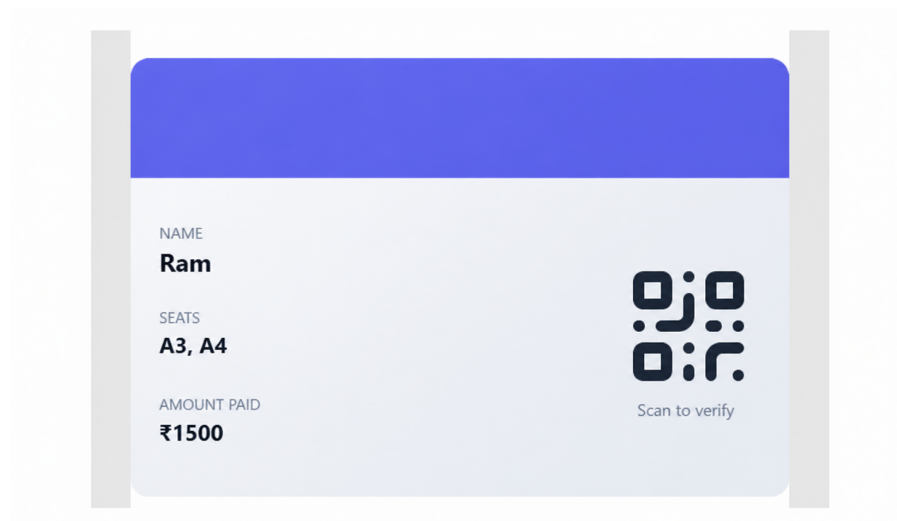


Figure 5.8: Ticket Pass

Fig 5.8 describes the ticket pass which is generated after successful payment, containing a unique QR code, event details, and participant information for verification at the venue.

Discussion

The Ticket Booking Web Application for Technical Fest proves to be an effective solution for managing technical fest events. Its modular architecture ensures scalability, security, and maintainability.

The integration of multiple modules like authentication, booking, payment, and notification improves overall efficiency and reduces manual work. The system enhances coordination between users and administrators while ensuring reliable and organized event management.

Table 5.1: Ticket Booking System Module Comparison

Modules	Features
Landing Page	User-friendly homepage
Authentication	Login and registration
Event Listing	Displays event details
Ticket Booking	Books tickets easily
Seat Selection	Selects available seats
Payment	Handles online transactions
E-Ticket / QR Code	Generates digital tickets
User Dashboard	Shows booked tickets
Admin Dashboard	Manages events and bookings
Notifications	Sends confirmation alerts
Reports	Shows booking insights

Table 5.1 describes the Ticket Booking System is divided into multiple modules that ensure smooth and organized functionality. It includes user-focused features like authentication, event listing, ticket booking, seat selection, and a dashboard to manage bookings.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Ticket Booking System for an Internal Department Event successfully provides a simple and efficient solution for managing event registrations. It reduces manual work, minimizes errors, and ensures smooth ticket booking through a user-friendly interface. The system improves overall event management by providing better control, accuracy, and convenience for both users and administrators.

6.2 Future Enhancements

While the current implementation of the Ticket Booking of an Internal Department Event system provides a strong foundation for managing events and bookings, the following enhancements are planned for future versions:

Online Payment Integration:

Integration of secure payment gateways to enable seamless online transactions, supporting multiple payment options such as credit/debit cards and digital wallets.

QR Code-Based Entry System:

Enhancing the ticket validation process by implementing QR code scanning at event entry points for faster and more secure check-ins.

Advanced Analytics Dashboard:

Introducing detailed analytics for administrators, including attendee insights, booking trends, and event performance metrics for better decision-making.

Email and SMS Notification System:

Automated alerts for booking confirmations, event reminders, cancellations, and updates to improve user engagement.

Role-Based Access Control Enhancement:

Expanding user roles (e.g., Super Admin, Organizer, Staff) to improve system security and provide better control over event management operations.

Seat Allocation Optimization:

Implementing intelligent seat allocation algorithms to optimize seating arrangements and enhance user convenience.

Chapter 7

SOURCE CODE

7.1 Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>TechFest Ticket Booking</title>

  <!-- React CDN -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/react/18.2.0/umd/react.development.js"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/react-dom/18.2.0/umd/react-dom.development.js"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/babel-standalone/6.26.0/babel.min.js"></script>

  <!-- jsPDF -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>

  <style>
    body { font-family: Arial; margin: 0; background:#f5f5f5; }
    .container { padding:20px; max-width:800px; margin:auto; }
    input, select, button {
      margin:8px 0;
      padding:10px;
      width:100%;
    }
    button { cursor:pointer; }
    .card { background:#fff; padding:20px; border-radius:10px; }
  </style>
</head>

<body>
<div id="root"></div>

<script type="text/babel">

const TOTAL_TICKETS = 100;
const PRICE = 100;

function App() {
```

```

const [form, setForm] = React.useState({
  name: "", email: "", phone: "", dept: "", qty: 1
});

const [booking, setBooking] = React.useState(null);
const [bookings, setBookings] = React.useState([]);

const available = TOTAL.TICKETS - bookings.reduce((s,b)=>s+b.qty,0);

const handleChange = (field, value) => {
  setForm({ ...form, [field]: value });
};

const submit = () => {

  if(!form.name || !form.email || !form.phone || !form.dept){
    alert("Fill all fields");
    return;
  }

  const newBooking = {
    ...form,
    ref: "REF" + Math.floor(Math.random()*10000),
    total: form.qty * PRICE,
    date: new Date().toLocaleString()
  };

  setBookings([newBooking, ...bookings]);
  setBooking(newBooking);
};

const downloadPDF = () => {
  const { jsPDF } = window.jspdf;
  const doc = new jsPDF();

  doc.text("TechFest Ticket", 20, 20);
  doc.text("Name:" + booking.name, 20, 40);
  doc.text("Tickets:" + booking.qty, 20, 50);
  doc.text("Total:Rs" + booking.total, 20, 60);
  doc.text("Ref:" + booking.ref, 20, 70);

  doc.save("ticket.pdf");
};

if(booking){
  return (
    <div className="container">
      <div className="card">
        <h2>Booking Confirmed </h2>

```



```

        <p>Name: {booking.name}</p>
        <p>Email: {booking.email}</p>
        <p>Tickets: {booking.qty}</p>
        <p>Total: {booking.total}</p>
        <p>Reference: {booking.ref}</p>

        <button onClick={downloadPDF}>Download PDF</button>
        <button onClick={()=>setBooking(null)}>Book More</button>
    </div>
</div>
);
}

return (
    <div className="container">
        <div className="card">
            <h2>TechFest 2025 Ticket Booking</h2>
            <p>Available Tickets: {available}</p>

            <input placeholder="Name"
                value={form.name}
                onChange={(e)=>handleChange("name", e.target.value)}
            />

            <input placeholder="Email"
                value={form.email}
                onChange={(e)=>handleChange("email", e.target.value)}
            />

            <input placeholder="Phone"
                value={form.phone}
                onChange={(e)=>handleChange("phone", e.target.value)}
            />

            <select value={form.dept}
                onChange={(e)=>handleChange("dept", e.target.value)}>
                <option value="">Select Department</option>
                <option>CSE</option>
                <option>ECE</option>
                <option>MECH</option>
            </select>

            <input type="number" min="1" max="10"
                value={form.qty}
                onChange={(e)=>handleChange("qty", Number(e.target.value))}
            />

            <button onClick={submit} disabled={available===0}>
                Confirm Booking
            </button>

```

```
        </div>
      </div>
    );
  }

ReactDOM.createRoot(document.getElementById("root")).render(<App/>);

</script>
</body>
</html>
```

Chapter 8

Addressing Complex Engineering Problem and SDG

8.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 8.1: Mapping of Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	Targeted WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering	WK4, WK5	Knowledge of React JS, hooks, validation, UI design
WP2	Conflicting Requirements	Technical and non-technical constraints	WK4, WK5	Balances usability with ticket booking validation
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Component design, state handling, dynamic updates
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking and validation logic
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Students, faculty, organizers
WP7	Interdependence	System-level integration	WK3, WK5	UI + state + validation integration

Table 8.1 describes the project addresses complex engineering problems by requiring in-depth technical knowledge, analytical thinking, and modern web development skills. It balances conflicting requirements such as usability and system validation while handling real-time ticket booking scenarios.

8.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 8.2: Mapping of Project to Complex Engineering Problem (CEP)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation
EA3	Innovation	Modern technologies	Hooks, conditional rendering
EA4	Consequences	Impact on users	Simplifies booking process
EA5	Familiarity	Beyond standard solutions	Dynamic real-time system

Table 8.2 describes that the project utilizes modern technologies and resources like React, HTML, and CSS to build an efficient system. It involves complex interactions between modules such as event display, booking, and validation.

8.3 Sustainable Development Goals (SDG) Mapping

Table 8.3: SDG Mapping of the Project

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic events and learning
SDG 9	Industry, Innovation and Infrastructure	Promotes modern React-based systems
SDG 10	Reduced Inequalities	Accessible platform for all users
SDG 11	Sustainable Cities and Communities	Encourages organized and efficient event management

Table 8.3 describes that the project aligns with multiple Sustainable Development Goals by supporting academic activities and enhancing access to educational events. It promotes innovation through the use of modern web technologies and provides an inclusive platform accessible to all users.

References

[1] Meta Platforms, Inc. (2024). *React: A JavaScript Library for Building User Interfaces*, React Official Documentation, 1(1), pp. 1–10.

<https://react.dev/>

[2] OpenJS Foundation (2024). *Node.js Runtime Environment Documentation*, Node.js Official Docs, 2(1), pp. 1–12.

<https://nodejs.org/en/docs>

[3] Oracle Corporation (2024). *MySQL Relational Database Management System*, MySQL Official Documentation, 3(2), pp. 1–15.

<https://dev.mysql.com/doc/>

[4] Razorpay Software Private Limited (2024). *Online Payment Gateway Integration for Web Applications*, Razorpay Developer Documentation, 5(1), pp. 10–18. **<https://razorpay.com/docs/>**

[5] Stripe, Inc. (2024). *Secure Payment Processing and API Integration*, Stripe API Reference, 6(2), pp. 20–30.

<https://stripe.com/docs/api>